

WEEK 10

TASK 1 - Running Sum of 1d Array

PROGRAM :

```
class Solution {  
    public int[] runningSum(int[] nums) {  
        int sum = 0;  
  
        for (int i = 0; i < nums.length; i++) {  
            sum += nums[i];  
            nums[i] = sum;  
        }  
  
        return nums;  
    }  
}
```

TASK 2 - Find All Numbers Disappeared in an Array

PROGRAM:

```
import java.util.*;  
  
class Solution {  
    public List<Integer> findDisappearedNumbers(int[] nums) {  
        List<Integer> result = new ArrayList<>();  
  
        for (int i = 0; i < nums.length; i++) {  
            int index = Math.abs(nums[i]) - 1;  
  
            if (nums[index] > 0) {
```

```

        nums[index] = -nums[index];
    }
}

for (int i = 0; i < nums.length; i++) {
    if (nums[i] > 0) {
        result.add(i + 1);
    }
}

return result;
}
}

```

TASK 3 - Sort Colors

PROGRAM:

```

class Solution {
    public void sortColors(int[] nums) {
        int low = 0;
        int mid = 0;
        int high = nums.length - 1;

        while (mid <= high) {
            if (nums[mid] == 0) {
                int temp = nums[low];
                nums[low] = nums[mid];
                nums[mid] = temp;
            }
        }
    }
}

```

```

        low++;
        mid++;
    }
    else if (nums[mid] == 1) {
        mid++;
    }
    else {
        int temp = nums[mid];
        nums[mid] = nums[high];
        nums[high] = temp;

        high--;
    }
}
}
}

```

TASK 4 - Design Linked List

PROGRAM:

```

class MyLinkedList {

    class Node {
        int val;
        Node next;

        Node(int val) {
            this.val = val;
            this.next = null;
        }
    }
}

```

```
Node head;
```

```
int size;
```

```
public MyLinkedList() {
```

```
    head = null;
```

```
    size = 0;
```

```
}
```

```
public int get(int index) {
```

```
    if (index < 0 || index >= size) {
```

```
        return -1;
```

```
    }
```

```
    Node current = head;
```

```
    for (int i = 0; i < index; i++) {
```

```
        current = current.next;
```

```
    }
```

```
    return current.val;
```

```
}
```

```
public void addAtHead(int val) {
```

```
    Node newNode = new Node(val);
```

```
    newNode.next = head;
```

```
    head = newNode;
```

```
    size++;
```

```
}
```

```
public void addAtTail(int val) {  
    Node newNode = new Node(val);  
  
    if (head == null) {  
        head = newNode;  
    } else {  
        Node current = head;  
  
        while (current.next != null) {  
            current = current.next;  
        }  
  
        current.next = newNode;  
    }  
  
    size++;  
}  
  
public void addAtIndex(int index, int val) {  
    if (index < 0 || index > size) {  
        return;  
    }  
  
    if (index == 0) {  
        addAtHead(val);  
        return;  
    }  
}
```

```
Node newNode = new Node(val);

Node current = head;

for (int i = 0; i < index - 1; i++) {
    current = current.next;
}

newNode.next = current.next;
current.next = newNode;

size++;
}

public void deleteAtIndex(int index) {
    if (index < 0 || index >= size) {
        return;
    }

    if (index == 0) {
        head = head.next;
        size--;
        return;
    }

    Node current = head;

    for (int i = 0; i < index - 1; i++) {
        current = current.next;
    }
```

```

        current.next = current.next.next;

        size--;
    }
}

```

TASK 5 - Min Stack

PROGRAM:

```

import java.util.Stack;

class MinStack {

    Stack<Integer> stack;
    Stack<Integer> minStack;

    public MinStack() {
        stack = new Stack<>();
        minStack = new Stack<>();
    }

    public void push(int val) {
        stack.push(val);

        if (minStack.isEmpty() || val <= minStack.peek()) {
            minStack.push(val);
        }
    }

    public void pop() {

```

```

        if (stack.peek().equals(minStack.peek())) {
            minStack.pop();
        }

        stack.pop();
    }

    public int top() {
        return stack.peek();
    }

    public int getMin() {
        return minStack.peek();
    }
}

```

TASK 6 - Merge Sorted Array

PROGRAM:

```

class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        int i = m - 1;
        int j = n - 1;
        int k = m + n - 1;

        while (i >= 0 && j >= 0) {
            if (nums1[i] > nums2[j]) {
                nums1[k] = nums1[i];
                i--;
            }
        }
    }
}

```

```

    } else {
        nums1[k] = nums2[j];
        j--;
    }
    k--;
}

while (j >= 0) {
    nums1[k] = nums2[j];
    j--;
    k--;
}
}
}

```

TASK 7 - Find Smallest Letter Greater Than Target

PROGRM:

```

class Solution {
    public char nextGreatestLetter(char[] letters, char target) {
        int left = 0;
        int right = letters.length - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (letters[mid] <= target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
    }
}

```

```

    }
}

// If no greater character exists, return the first character
return letters[left % letters.length];
}
}

```

TASK 8 - Valid Parentheses

PROGRAM:

```

import java.util.Stack;

class Solution {
    public boolean isValid(String s) {
        Stack<Character> stack = new Stack<>();

        for (char ch : s.toCharArray()) {

            if (ch == '(' || ch == '[' || ch == '{') {
                stack.push(ch);
            }
            else {
                if (stack.isEmpty()) {
                    return false;
                }

                char top = stack.pop();

                if ((ch == ')' && top != '(') ||

```

```

        (ch == ']' && top != '[') ||
        (ch == '}' && top != '{')) {
            return false;
        }
    }
}

return stack.isEmpty();
}
}

```

TASK 9 - Implement Queue using Stacks

PROGRAM:

```

import java.util.Stack;

class MyQueue {

    Stack<Integer> input;
    Stack<Integer> output;

    public MyQueue() {
        input = new Stack<>();
        output = new Stack<>();
    }

    public void push(int x) {
        input.push(x);
    }
}

```

```
public int pop() {  
    moveElements();  
    return output.pop();  
}
```

```
public int peek() {  
    moveElements();  
    return output.peek();  
}
```

```
public boolean empty() {  
    return input.isEmpty() && output.isEmpty();  
}
```

```
private void moveElements() {  
    if (output.isEmpty()) {  
        while (!input.isEmpty()) {  
            output.push(input.pop());  
        }  
    }  
}  
}
```